PROJECT DOCUMENT

EDUGENIE:GOOGLE GEMINI POWERED LEARNING ASSISTANT

TEAM ID:**SWTID-2026-9532**

TEAM SIZE:**5**

TEAM LEADER:**Divya Dharshini**

EMAIL ID: meenadavid20082007@gmail.com

TEAM MEMBER:**R.Vindhiya**

EMAIL ID:rameshvindhiya9@gmail.com

TEAM MEMBER:**J.Sandhiya**

EMAIL ID:vijaytharun9940@gmail.com

TEAM MEMBER:**P.Sandhiya**

EMAIL ID:sandhiyasandhiya26547@gamil.com

TEAM MEMBER:**D.Sowjanya**

EMAIL ID:sowjanyad400@gmail.com

EduGenie: Google Gemini Powered Learning Assistant

PHASE 1: PROJECT OVERVIEW AND LEARNING SCENARIOS

Project Description

EduGenie is an AI-supported learning platform that helps students organize study tasks, understand difficult lessons, practise questions, and revise important information. The application combines a web interface with a backend service so that learners can access several study functions from one place.

- Ask subject-related questions and receive focused answers.
- Break difficult lessons into simple explanations with examples.
- Create practice questions from a topic, lesson, or supplied passage.
- Build a structured study route from basic concepts to advanced topics.
- Convert lengthy notes into concise revision material.

The application uses FastAPI for backend request handling and HTML/CSS for the learner interface. AI services are connected through dedicated modules so that each learning task can be maintained and tested independently.

Scenarios

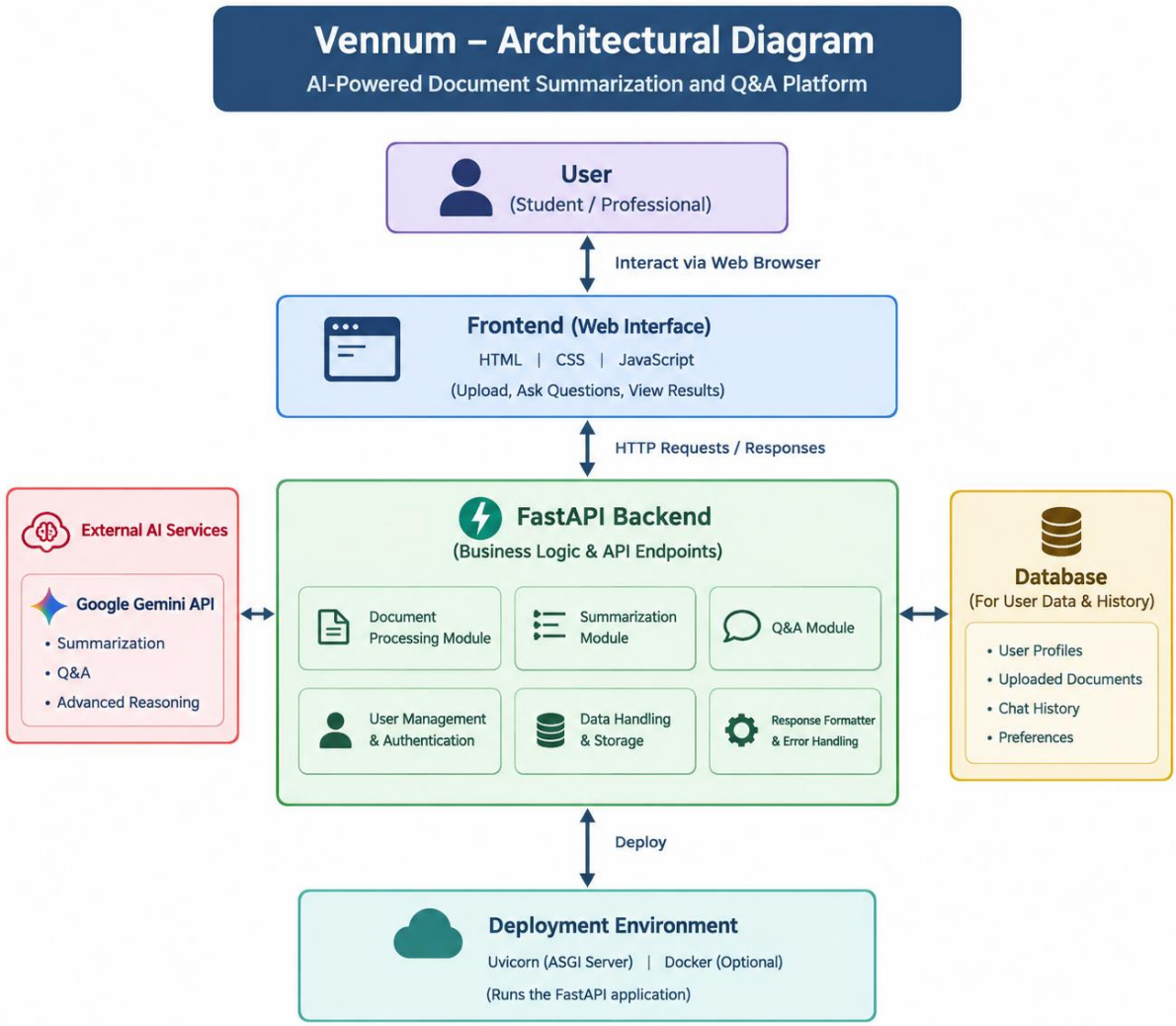
Scenario 1: A student studying the water cycle asks EduGenie to explain evaporation, condensation, and precipitation in simple terms.

Scenario 2: A student revising a science chapter enters the lesson and uses the quiz feature to practise before a class test.

Scenario 3: A beginner learning database concepts requests a step-by-step path covering tables, keys, queries, joins, and advanced SQL topics.

PHASE 2: SYSTEM ARCHITECTURE AND PREREQUISITES

ARCHITECTURAL DIAGRAM



Pre-requisites

1. **Python 3.10+**

- Official Documentation: Python Documentation
- Installation Guide: Python Software Foundation
- Popular Tutorial: Python Programming for Beginners
- **Installation Steps:**

    1. Download a supported Python 3.10+ installer for the operating system.
    2. Install Python and enable the option to add it to the system PATH.
    3. Complete the installation and open a terminal to confirm that Python is available.

4. Run python --version to verify the installed version.

2. **FastAPI Framework –**

- Official Documentation: FastAPI Documentation
- User Guide: FastAPI Tutorial and Reference
- Popular Tutorial: Building APIs with FastAPI

3. **HTML & CSS –** Used to create the page structure, controls, and visual presentation of the learning interface.

4. **Google Gemini API Key**

- Official Documentation: Google AI for Developers
- Setup Guide: Google AI Studio
- Popular Tutorial: Gemini API Integration
- Setup Steps:
    1. Open the Google AI development environment and sign in with an authorized account.
    2. Create or select a project and enable the required Gemini service.
    3. Create an API key and store it securely outside the public frontend.

5. **Uvicorn (ASGI Server)**

- Official Documentation: PyPI
- Installation Steps:
    1. Install Uvicorn using pip: pip install uvicorn

6. **Jinja2 (HTML Templating Engine)**

- Official Documentation: Jinja2 Documentation
- Installation Steps:
    1. Install Jinja2 using pip: pip install jinja2

Project Workflow

PHASE 3: AI MODEL SELECTION AND PROJECT STRUCTURE

MILESTONE 1: Model Selection and Architecture

Activity 1.1: Select AI Models

- Gemini model (cloud service): Used for question answering, quiz generation, summarization, and study recommendations. It provides natural-language understanding and flexible responses for different learning tasks.

  Lightweight local model: Used where a compact explanation component is required.

It can provide short concept descriptions while reducing dependence on repeated cloud requests.

- **LaMini-Flan-T5-783M (local):**
  Used for: Concept explanation
  Benefits: Instruction-tuned, lightweight, CPU-compatible

Folder Architecture:

EduGenie/
1. main.py # FastAPI app
2. explanation_module.py # Concept explanation logic
3. qna.py # Question answering
4. quiz_module.py # Quiz generation
5. summary_module.py # Summarization
6. learning_path.py # Learning recommendations
7. templates/index.html # HTML frontend
8. static/style.css # Styling
9. requirements.txt # Python dependencies

Folder Architecture:

EduGenie/
1. main.py # FastAPI application entry point
2. explanation_module.py # Concept explanation logic
3. qna.py # Academic question handling
4. quiz_module.py # Practice question generation
5. summary_module.py # Revision summary generation
6. learning_path.py # Study route recommendations
7. templates/index.html # Web page structure
8. static/style.css # Interface design
9. requirements.txt # Required Python packages

FIG. Revised Folder Architecture

PHASE 4: CORE LEARNING MODULES

MILESTONE 2: Core Functionalities Development

Activity 2.1. : Module Implementation

1.Explanation Module

The Explanation Module receives a topic and prepares a learner-oriented response. Instead of presenting a dense definition, it can organize the answer into a short meaning, key points, an example, and a simple recap. This structure helps students move from basic understanding to clearer recall.

```
planation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
```

Fig. Revised Explanation Module Flow

2.QnA Module

The QnA Module accepts a student's question, sends the request through the backend, and returns a focused response. It is designed for quick doubt clarification and can handle questions from different academic areas while keeping the answer aligned with the learner's input.

```
def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in QnA: {e}"
```

Fig. Revised Q/A Module Flow

3.Quiz Module

The Quiz Module converts supplied study material into multiple-choice practice questions. The module identifies important concepts, creates questions with four choices, and marks the correct option. The structured output allows the frontend to present the quiz in a consistent format.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")

        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧼 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
```

Fig. Revised Quiz Module Logic

A structured prompt is sent to the AI service so that every question follows the expected format. The backend validates the returned data before displaying it. If the response is incomplete or cannot be parsed, the module returns a controlled error message rather than displaying broken quiz data.

4.Summary Module

The Summary Module extracts the central ideas from a long passage and rewrites them in a shorter form. It is intended for revision, allowing students to review definitions, processes, and important facts without reading the complete source every time.

```python
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Fig. Revised Summary Module Flow

5.Learning Path Module

The get_learning_recommendations function creates a study sequence for a selected subject. It can divide a broad topic into beginner, intermediate, and advanced stages, while suggesting what concept should be learned before the next one. The module also handles incomplete responses and reports errors clearly.

```python
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🧠 Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc()  # ✅ This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```

Fig. Revised Learning Path Flow

PHASE 5: BACKEND API DEVELOPMENT

Activity 2.2 : Backend API with FastAPI

- Defined RESTful endpoints for the major learning functions:

- /qa
- /explain
- /quiz
- /summarize
- /learn/recommendations

- Each endpoint is connected to one dedicated module so that input validation, AI processing, and response formatting remain separated.

main.py

```python

# Q&A - GET API using Gemini
@app.get("/qa")
async def answer_question(question: str = Query(...)):
    answer = answer_question_with_gemini(question)
    return {"answer": answer}

# Explanation - POST API
@app.post("/explain/")
async def explain_api(request: Request):
    data = await request.json()
    topic = data.get("topic")
    if not topic:
        return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
    explanation = explain_topic(topic)
    return {"topic": topic, "explanation": explanation}

# Summarization - POST API
@app.post("/summarize/")
async def summarize_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
    summary = summarize_text(text)
    return {"summary": summary}

```

Fig. Revised API Endpoint Flow

```python

# Quiz Generation - POST API
@app.post("/quiz")
async def quiz_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
    quiz = generate_quiz(text)
    print("Generated quiz:", quiz)  # ✅ DEBUG
    return JSONResponse(content={"quiz": quiz})

# Learning Recommendations - GET API
@app.get("/learn/recommendations")
async def learning_recommendation_api(topic: str = Query(...)):
    recommendation = get_learning_recommendations(topic)
    return {"topic": topic, "recommendation": recommendation}

```

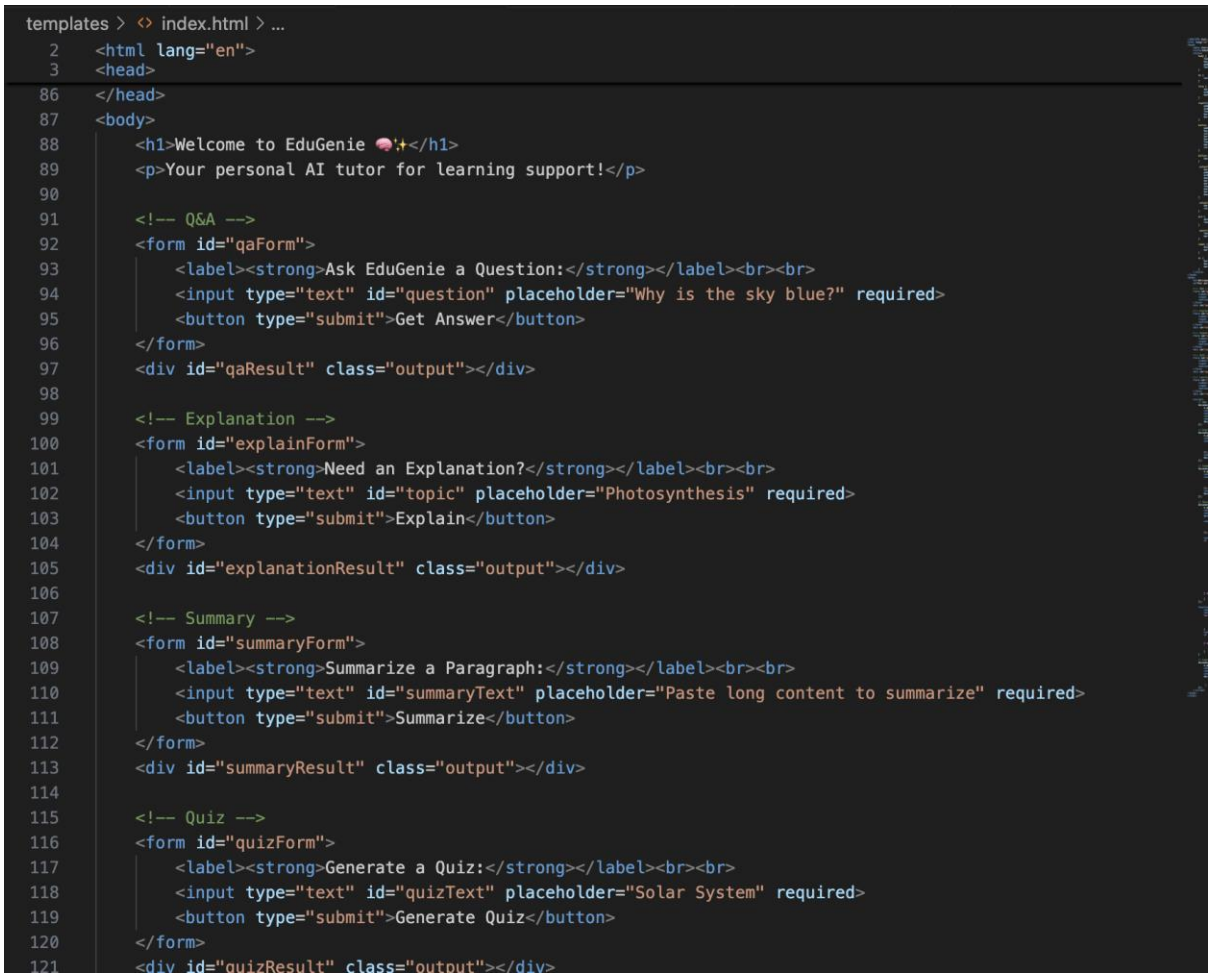
Fig. Revised API Request Cycle

PHASE 6: FRONTEND DEVELOPMENT AND INTEGRATION

MILESTONE 3: Frontend Development

Activity 3.1: Build Web Interface

- **HTML Form (index.html):**
    - Task dropdown (Explain, QnA, Quiz, Summary, Recommend Path)
    - Text area for entering a question, topic, or study passage
    - Submit button to send the selected task
- **CSS Styling (style.css):**
    - Responsive design for different screen sizes
    - Clear buttons, input fields, headings, and result containers for easy navigation



templates > index.html > ...

```
<html lang="en">
<head>
</head>
<body>
    <h1>Welcome to EduGenie 🧠✨</h1>
    <p>Your personal AI tutor for learning support!</p>

    <!-- Q&A -->
    <form id="qaForm">
        <label><strong>Ask EduGenie a Question:</strong></label><br><br>
        <input type="text" id="question" placeholder="Why is the sky blue?" required>
        <button type="submit">Get Answer</button>
    </form>
    <div id="qaResult" class="output"></div>

    <!-- Explanation -->
    <form id="explainForm">
        <label><strong>Need an Explanation?</strong></label><br><br>
        <input type="text" id="topic" placeholder="Photosynthesis" required>
        <button type="submit">Explain</button>
    </form>
    <div id="explanationResult" class="output"></div>

    <!-- Summary -->
    <form id="summaryForm">
        <label><strong>Summarize a Paragraph:</strong></label><br><br>
        <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
        <button type="submit">Summarize</button>
    </form>
    <div id="summaryResult" class="output"></div>

    <!-- Quiz -->
    <form id="quizForm">
        <label><strong>Generate a Quiz:</strong></label><br><br>
        <input type="text" id="quizText" placeholder="Solar System" required>
        <button type="submit">Generate Quiz</button>
    </form>
    <div id="quizResult" class="output"></div>
```

Fig. Revised Frontend Integration

Activity 3.2: Live Integration

- The form sends the selected request to the FastAPI backend. The backend calls the appropriate module and returns the generated result to the frontend.
- The completed response is displayed in the result section so the learner can continue studying without manually opening another tool.

PHASE 7: DEPLOYMENT AND FUNCTIONAL TESTING

MILESTONE 4: Deployment

- **Activity 4.1:** Run Locally
  Run locally with uvicorn main:app --reload
  Navigate to: http://127.0.0.1:8000

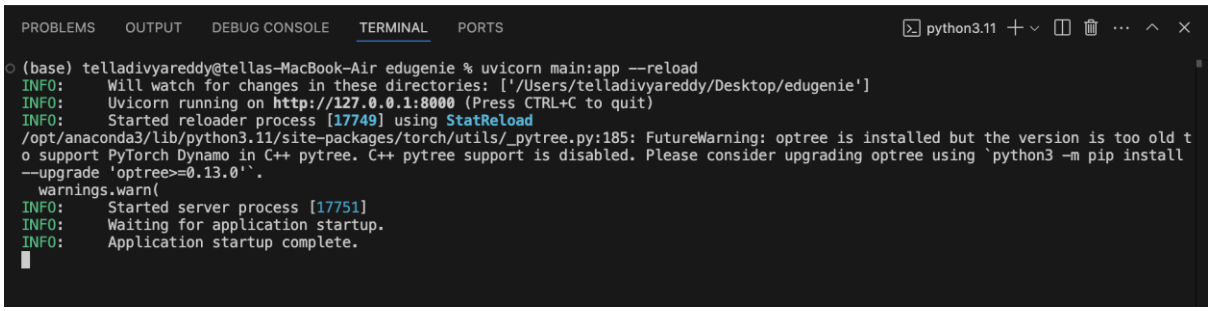


Fig. Revised Local Deployment Flow

- **Activity 4.2:** Functional Testing
  Ask an academic question
  Request a simple explanation
  Generate practice questions
  Summarize study material
  Create a step-by-step learning plan

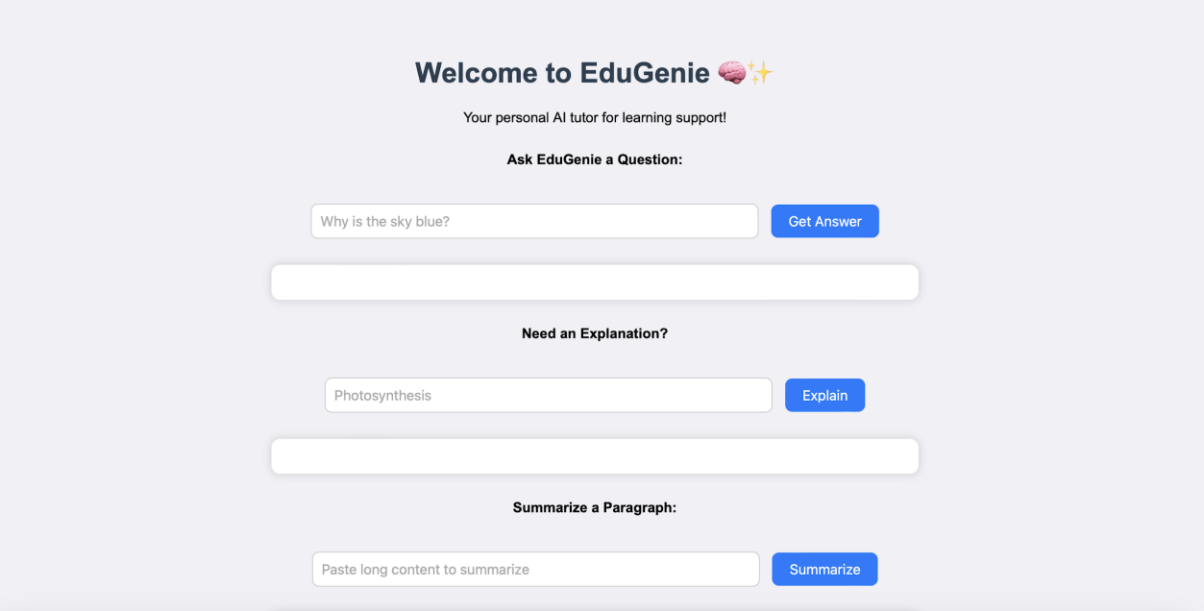


Fig. Revised EduGenie Dashboard

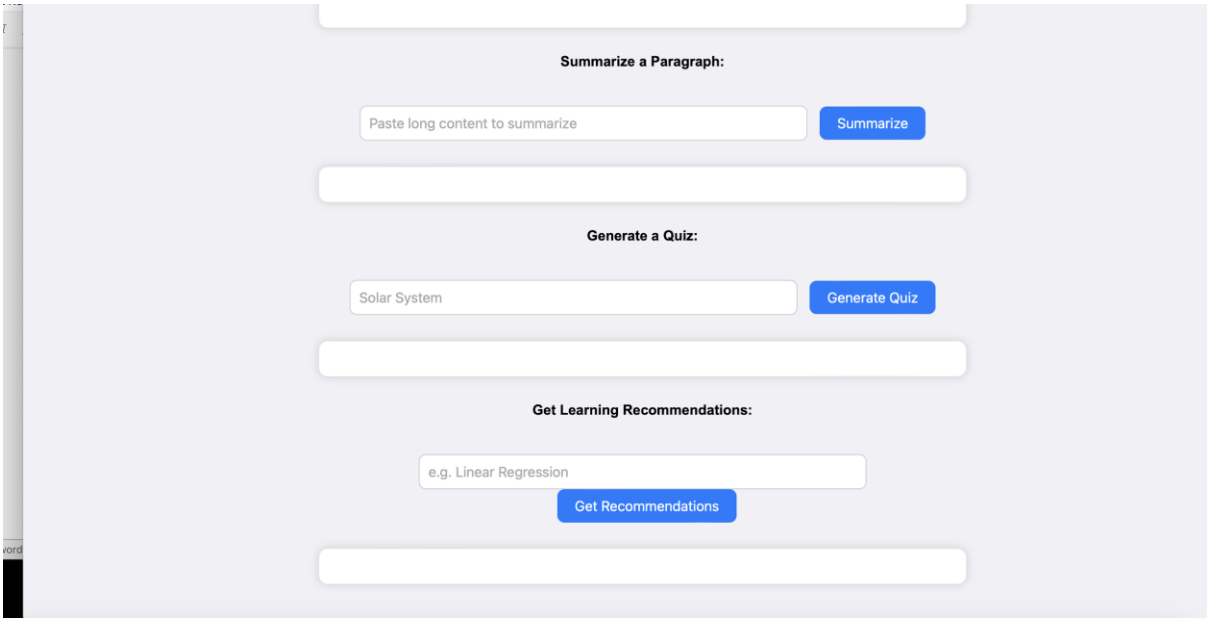


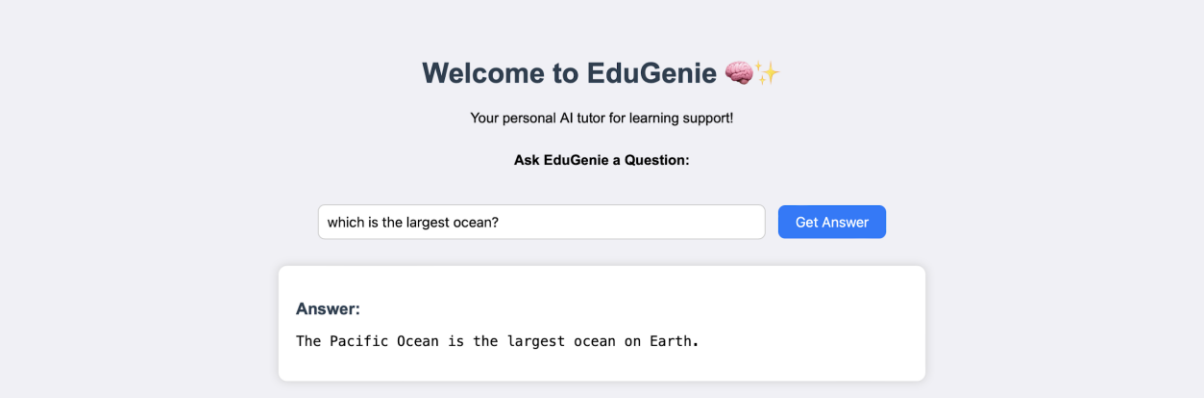
Fig. Revised EduGenie Learning Screen

PHASE 8: EXPLORING EDUGENIE AND CONCLUSION

Exploring EduGenie:

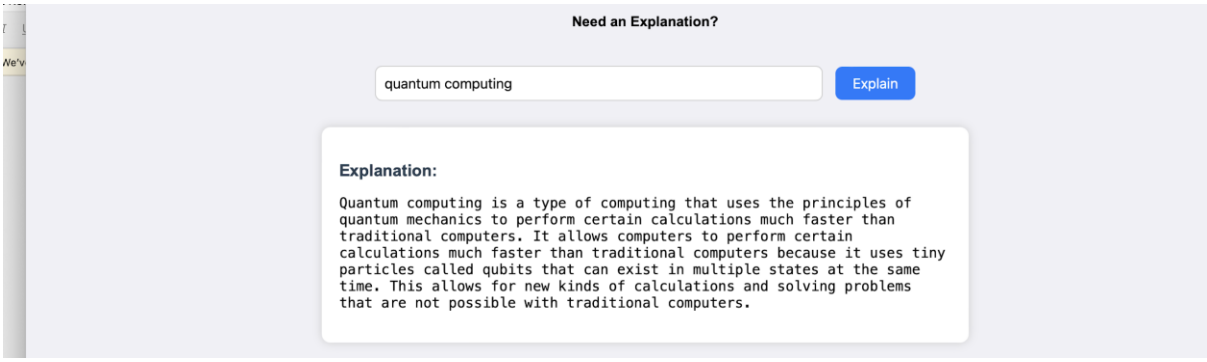
A.Asking questions:

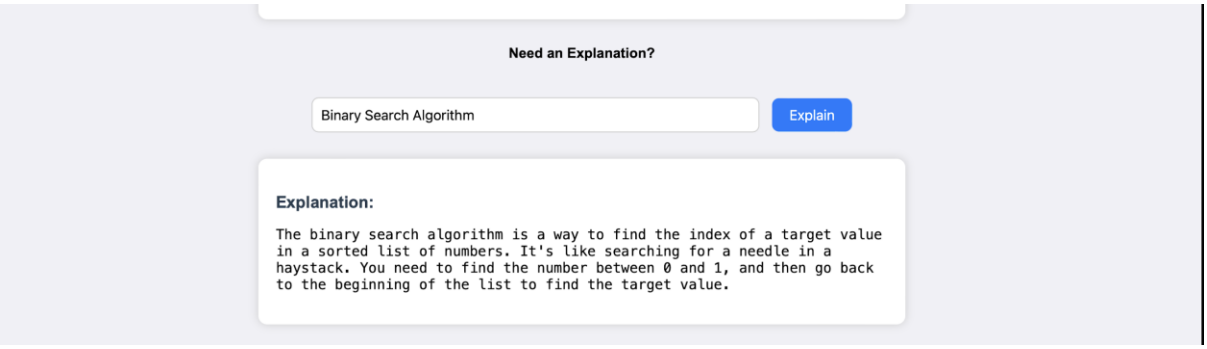
Students can enter a subject question in the input area. EduGenie processes the request and presents a focused response that can be used as a starting point for further study.



b.Explanation of any topic:

The explanation function presents difficult concepts in simpler language and can organize the response into definitions, examples, and key points.



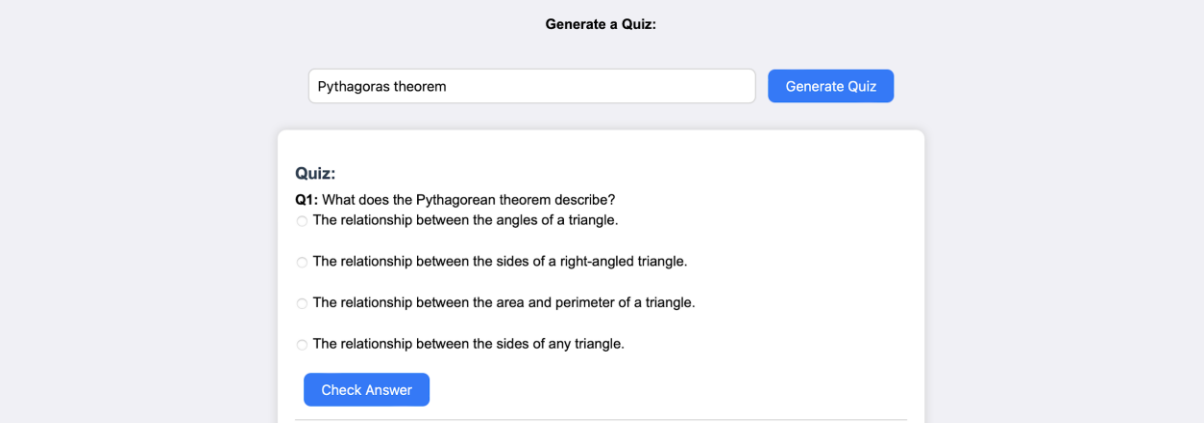


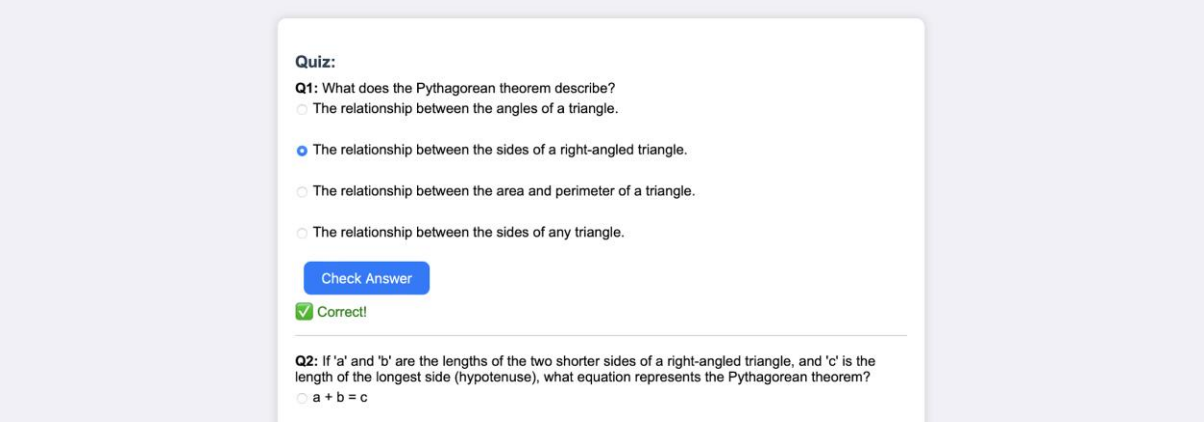
C.Summarising long paragraphs:

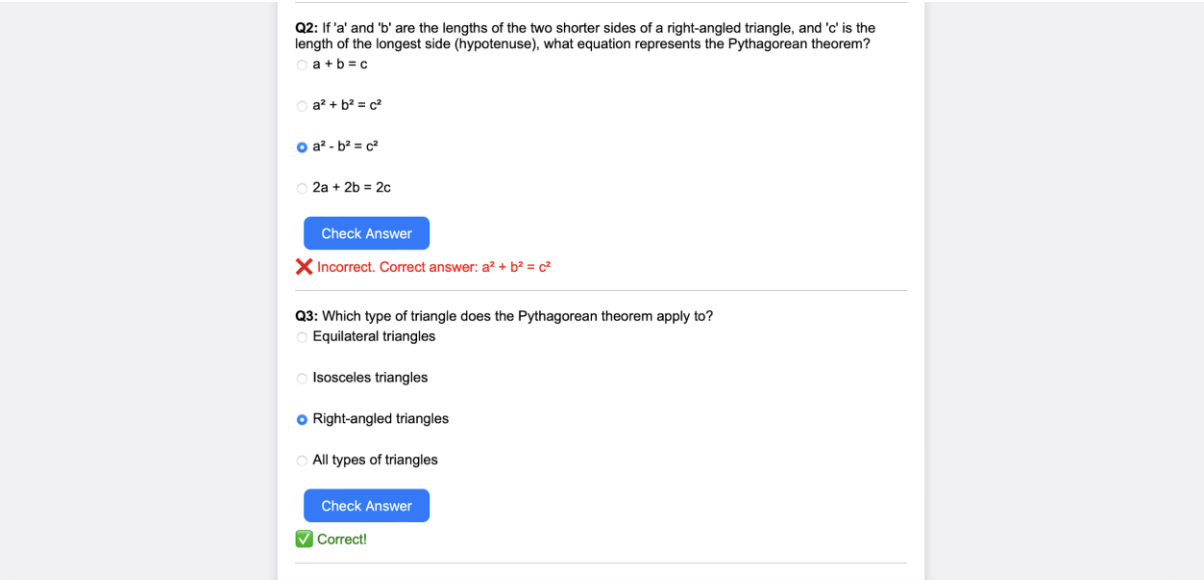
Learners can paste lengthy notes or textbook passages into the summary feature. The generated output focuses on the central information and removes unnecessary repetition.

D.Generating quizzes:

The quiz feature creates three practice questions with four options each. The learner can use the questions to identify which parts of the lesson need further revision.







E.Guiding with learning recommendations from beginners level to advanced level

The learning-path feature organizes a subject into a progressive sequence. It starts with prerequisite concepts, moves through core topics, and then introduces more advanced areas.

```
complex topics.  It's designed to be adaptive: feel free to adjust the
pace and delve deeper into areas that particularly interest you.

**I. Beginner Level: Building a Foundation**

**(Estimated Time: 1-2 weeks)**

* **Key Topics:**
    * What is a Database and SQL? (Relational Model, DBMS)
    * Basic Syntax (SELECT, FROM, WHERE)
    * Data Types (INT, VARCHAR, DATE, etc.)
    * Filtering Data (WHERE clause with comparison operators, logical
operators: AND, OR, NOT)
    * Ordering Results (ORDER BY)
    * Limiting Results (LIMIT/OFFSET or TOP)
    * Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
    * Introduction to Tables and Columns

* **Resources:**
    * **Interactive Tutorials:**
        * SQLZoo: Excellent for hands-on practice with different SQL
flavors.
        * Khan Academy's SQL course: Offers a good introductory
overview.
        * Codecademy's Learn SQL: Interactive lessons with practical
exercises.
    * **Videos:**
        * FreeCodeCamp's SQL Tutorial: Comprehensive and beginner-
friendly.


**II. Intermediate Level: Working with Multiple Tables**

**(Estimated Time: 2-3 weeks)**
```

```
**II. Intermediate Level: Working with Multiple Tables**

**(Estimated Time: 2-3 weeks)**

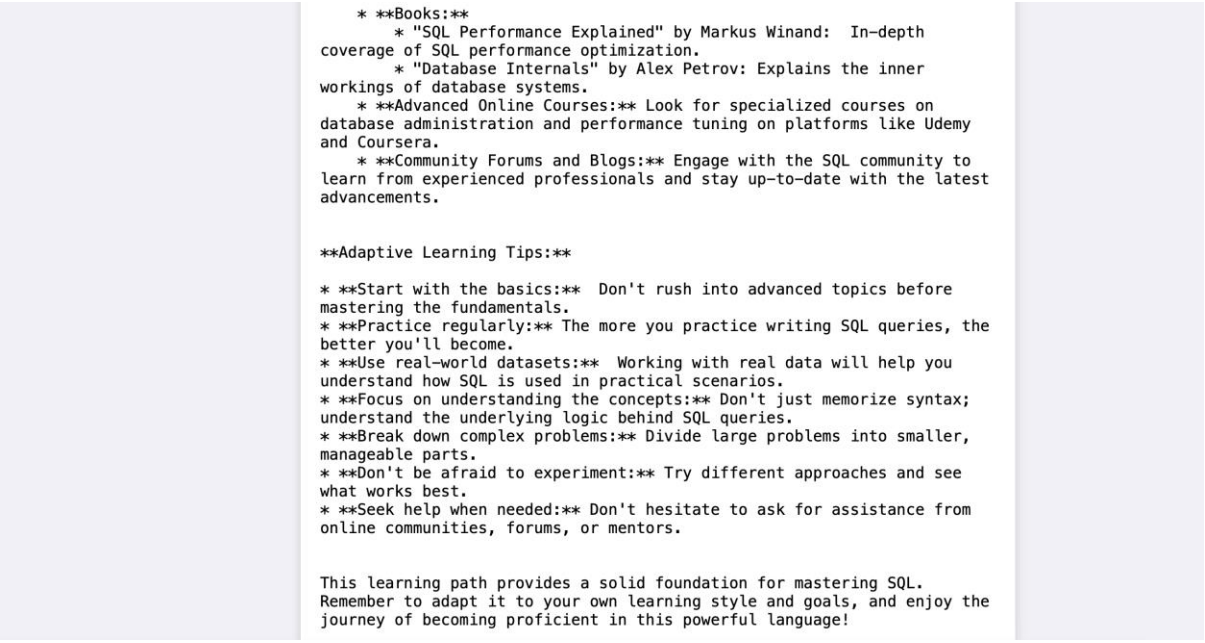
* **Key Topics:**
    * Joining Tables (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER
JOIN)
    * Subqueries (Nested queries)
    * Grouping Data (GROUP BY)
    * Filtering Groups (HAVING)
    * Set Operations (UNION, INTERSECT, EXCEPT)
    * Working with Strings (string concatenation, substrings, LIKE
operator)
    * Working with Dates and Times (date formatting, date arithmetic)
    * Views (Creating and using views)

* **Resources:**
    * **Books:**
        * "SQL Queries for Mere Mortals" by Michael J. Hernandez: A
great resource for understanding practical SQL querying.
    * **Online Documentation:** Refer to the documentation for your
specific SQL database (e.g., MySQL, PostgreSQL, SQL Server).
    * **Practice Platforms:** LeetCode, HackerRank, and similar
platforms offer SQL challenges to reinforce your skills.

**III. Advanced Level: Mastering Database Management**

**(Estimated Time: 3-4 weeks and beyond)**

* **Key Topics:**
    * Stored Procedures and Functions
    * Triggers
    * Indexes and Performance Tuning
    * Transactions and Concurrency Control
    * Database Design (Normalization)
```



```
    * **Books:**
        * "SQL Performance Explained" by Markus Winand:  In-depth
coverage of SQL performance optimization.
        * "Database Internals" by Alex Petrov: Explains the inner
workings of database systems.
    * **Advanced Online Courses:** Look for specialized courses on
database administration and performance tuning on platforms like Udemy
and Coursera.
    * **Community Forums and Blogs:** Engage with the SQL community to
learn from experienced professionals and stay up-to-date with the latest
advancements.


**Adaptive Learning Tips:**

* **Start with the basics:**  Don't rush into advanced topics before
mastering the fundamentals.
* **Practice regularly:** The more you practice writing SQL queries, the
better you'll become.
* **Use real-world datasets:**  Working with real data will help you
understand how SQL is used in practical scenarios.
* **Focus on understanding the concepts:** Don't just memorize syntax;
understand the underlying logic behind SQL queries.
* **Break down complex problems:** Divide large problems into smaller,
manageable parts.
* **Don't be afraid to experiment:** Try different approaches and see
what works best.
* **Seek help when needed:** Don't hesitate to ask for assistance from
online communities, forums, or mentors.


This learning path provides a solid foundation for mastering SQL.
Remember to adapt it to your own learning style and goals, and enjoy the
journey of becoming proficient in this powerful language!
```

It can also provide resource categories and stepwise study guidance, helping the learner understand what to learn first, what to practise next, and which advanced concepts can be attempted later.

Conclusion:

EduGenie brings question answering, concept explanation, quiz practice, summarization, and learning guidance together in one educational web application. Its modular structure separates the major learning functions, making the system easier to test, maintain, and extend.

The project demonstrates how a simple frontend can communicate with a FastAPI backend and specialized AI modules. Each module has a defined responsibility, while the backend provides a common route for receiving requests and returning results.

During development, important challenges include validating AI-generated output, maintaining consistent response formats, handling failed requests, and keeping the interface easy to use. These areas require repeated testing and careful prompt and response design.

Future improvements can include multilingual learning support, voice-based interaction, richer question types, and improved personalization based on a learner's study history.

A progress dashboard could record completed topics, quiz attempts, revision activity, and recommended next steps. This would give learners a clearer view of their study process.

The platform could also support teacher-created content, classroom groups, image and PDF input, and integration with learning management systems. Such features would extend the system beyond individual revision.

Overall, EduGenie provides a practical foundation for an interactive learning assistant. By combining a simple interface with modular AI services, it can support students in understanding concepts, practising questions, revising content, and planning their learning journey.